

RL and Imitation Learning on PushT Environment

Xiyuan Yang
524531910015

June 2026

0.1 PushT Environment

PushT is a 2D manipulation task where an agent must push a T-shaped block into a goal zone. The environment provides a low-dimensional state observation and a continuous action space.

Observation. The environment state is a 5-dimensional vector:

$$\mathbf{o} = [x_{\text{agent}}, y_{\text{agent}}, x_{\text{block}}, y_{\text{block}}, \theta_{\text{block}}] \in [0, 512]^4 \times [0, 2\pi].$$

The first two dimensions are the agent’s position; the last three encode the block’s position and rotation angle.

Action. The raw action space is a 2D target position $\mathbf{a}_{\text{raw}} = [x, y] \in [0, 512]^2$. We use a *delta action* wrapper that converts the policy output $\hat{\mathbf{a}} \in [-1, 1]^2$ into a relative displacement:

$$\mathbf{a}_{\text{raw}} = \text{clip}(\mathbf{p}_{\text{agent}} + \delta_{\text{max}} \cdot \hat{\mathbf{a}}, 0, 512),$$

where $\mathbf{p}_{\text{agent}}$ is the current agent position and $\delta_{\text{max}} = 100$ controls the maximum displacement per step.

Reward. The environment reward is based on the coverage ratio $c \in [0, 1]$ of the block within the goal zone:

$$r_{\text{env}} = \text{clip}(c/0.95, 0, 1).$$

We optionally apply a reward shaping wrapper that augments this signal:

$$r_{\text{shaped}} = \begin{cases} 20 & \text{if } c \geq 0.95 \text{ (success),} \\ 1 + 10 \cdot \Delta c & \text{if the block moved,} \\ 0 & \text{otherwise,} \end{cases}$$

where $\Delta c = c_t - c_{t-1}$ is the coverage change. This provides denser feedback for exploration.

Observation normalization. We normalize the observation by dividing by the known bounds:

$$\tilde{\mathbf{o}} = \mathbf{o} \oslash [512, 512, 512, 512, 2\pi],$$

mapping each component approximately to $[0, 1]$. This stabilizes training for both RL and BC.

Episode. Each episode lasts up to 300 steps (30 seconds at 10 FPS). Success requires $\geq 95\%$ coverage.

1 Problem 1: Pure RL on PushT

1.1 Algorithm Implementations

We implement two standard continuous-control RL algorithms: PPO (on-policy) and SAC (off-policy). Both use the same network architecture for fair comparison.

1.1.1 PPO (Proximal Policy Optimization)

PPO is an on-policy actor-critic method. The actor π_θ is a Gaussian policy with tanh squashing:

$$\mathbf{a} = \tanh(\mathbf{z}), \quad \mathbf{z} \sim \mathcal{N}(\mu_\theta(\mathbf{o}), \sigma_\theta^2),$$

where μ_θ is the mean network and σ_θ is a learnable log-standard-deviation parameter. The log-probability includes the tanh correction:

$$\log \pi_\theta(\mathbf{a} \mid \mathbf{o}) = \log \mathcal{N}(\mathbf{z} \mid \mu, \sigma) - \sum_i [2 \log 2 - z_i - \text{softplus}(-2z_i)].$$

The critic $V_\phi(\mathbf{o})$ estimates the state value. We use Generalized Advantage Estimation (GAE):

$$\hat{A}_t = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V_\phi(\mathbf{o}_{t+1}) - V_\phi(\mathbf{o}_t),$$

with discount $\gamma = 0.99$ and $\lambda = 0.95$. The clipped surrogate loss is:

$$L^{\text{CLIP}}(\theta) = \mathbb{E} \left[\min \left(\frac{\pi_\theta}{\pi_{\theta_{\text{old}}}} \hat{A}, \text{clip} \left(\frac{\pi_\theta}{\pi_{\theta_{\text{old}}}}, 1-\epsilon, 1+\epsilon \right) \hat{A} \right) \right],$$

with clip coefficient $\epsilon = 0.2$. The total loss combines policy, value, and entropy terms:

$$L = L^{\text{CLIP}} + c_v L^{\text{VF}} - c_e \mathcal{H}[\pi_\theta],$$

with $c_v = 1.0$ and $c_e = 0.005$. Gradients are clipped at norm 0.5.

1.1.2 SAC (Soft Actor-Critic)

SAC is an off-policy method that maximizes a trade-off between reward and entropy:

$$J(\pi) = \sum_t \mathbb{E} [r_t + \alpha \mathcal{H}[\pi_\theta(\cdot \mid \mathbf{o}_t)]],$$

where α is the temperature parameter, automatically tuned to match a target entropy $\bar{\mathcal{H}} = -2$:

$$\alpha \leftarrow \alpha - \eta_\alpha \nabla_\alpha \mathbb{E} [-\alpha (\log \pi_\theta + \bar{\mathcal{H}})].$$

The actor outputs a mean and log-std, applies tanh squashing, and uses the reparameterization trick for gradient estimation. Twin Q-networks Q_{ψ_1}, Q_{ψ_2} mitigate overestimation:

$$Q_{\text{target}} = r + \gamma(1-d) \left(\min_i Q_{\bar{\psi}_i}(\mathbf{o}', \mathbf{a}') - \alpha \log \pi_\theta(\mathbf{a}' \mid \mathbf{o}') \right).$$

Target networks are updated via Polyak averaging: $\bar{\psi} \leftarrow \tau \psi + (1-\tau) \bar{\psi}$ with $\tau = 0.005$. The replay buffer stores 1M transitions; batches of 2048 are sampled uniformly.

1.1.3 Practical Features

On top of the base algorithms, we implement the following features:

- **Delta action space:** The policy predicts relative displacements $\hat{\mathbf{a}} \in [-1, 1]^2$, scaled by $\delta_{\text{max}} = 100$ and added to the current agent position. This is easier to learn than absolute positioning.
- **Reward shaping:** A touch bonus (+1 when coverage changes) and coverage delta reward ($10 \cdot \Delta c$) provide denser feedback than the sparse environment reward.
- **Observation normalization:** Fixed division by $[512, 512, 512, 512, 2\pi]$ maps inputs to $\sim [0, 1]$.
- **Curriculum learning** (not used in final ablations): Starts with the block near the goal and gradually expands to full random placement.

1.1.4 Hyperparameters

All RL experiments train for 1M environment steps with seed 42. The shared and algorithm-specific hyperparameters are listed in Table 1.

Table 1: RL hyperparameters.

Parameter	PPO	SAC
Network	4-layer MLP, hidden dim 1024, Tanh	
Learning rate	3×10^{-4}	3×10^{-4} (actor, critic, α)
Discount γ	0.99	0.99
Batch size	2048	2048
Num environments	64	1
Rollout steps	512	—
Epochs per update	4	—
GAE λ	0.95	—
Clip ϵ	0.2	—
Entropy coeff c_e	0.005	auto ($\bar{\mathcal{H}} = -2$)
Value coeff c_v	1.0	—
Grad clip norm	0.5	∞
Replay buffer size	—	1,000,000
Warmup steps	—	1,000
Target τ	—	0.005
Action space	Delta ($\delta_{\max} = 100$)	
Reward shaping	ON	
Obs normalization	ON	
Eval frequency	every 2,000 steps	
Eval episodes	10	

The network uses orthogonal initialization with Tanh activations. The actor outputs actions in $[-1, 1]$ via tanh squashing. PPO uses 64 parallel environments for data collection, while SAC uses a single environment with a large replay buffer.

1.1.5 Analysis Report

Table 2 compares the best evaluation results of PPO and SAC after 1M training steps. The evaluation uses deterministic actions (mean of the policy distribution) over 10 episodes.

Table 2: Best RL evaluation results over 1M training steps (seed 42, 10 evaluation episodes, deterministic policy).

Algorithm	Best Eval R	Final Eval R	Ep Reward	Ep Len	Entropy	FPS
PPO	50.2	23.5	35.5	282	2.33	669
SAC	292.9	281.5	256.9	300	auto	38

SAC dramatically outperforms PPO, achieving $\sim 6\times$ higher best evaluation reward (292.9 vs 50.2) and maintaining high performance throughout training (final 281.5 vs 23.5). Several factors explain this gap:

1. **Sample efficiency:** SAC’s replay buffer (1M transitions) reuses past experience, while PPO discards data after each on-policy update. For a contact-rich task like PushT, successful manipulation trajectories are rare and valuable.

2. **Exploration:** SAC’s entropy regularization encourages broader exploration, helping discover the block early in training. PPO’s on-policy nature limits exploration diversity.
3. **Training stability:** PPO’s performance degrades over time (best 50.2 at 393K steps, final 23.5 at 1M steps), suggesting catastrophic forgetting or policy collapse. SAC maintains stable performance throughout.
4. **Value estimation:** SAC’s twin Q-networks with target networks provide more stable value estimates than PPO’s single value function with GAE.

1.2 Ablation Results

We run four ablation groups, each for 1M steps with the same network architecture, to isolate the effect of individual design choices:

1. **Algorithm:** PPO vs SAC (on-policy vs off-policy).
2. **Action space:** Absolute positioning vs delta (relative) positioning.
3. **Reward shaping:** With vs without the additional touch bonus and coverage delta reward.
4. **Observation normalization:** With vs without fixed normalization of the 5D state.

1.2.1 Ablation 1: Algorithm (PPO vs SAC)

This ablation compares on-policy (PPO) and off-policy (SAC) learning under identical conditions: delta actions, reward shaping ON, observation normalization ON, and the same 4-layer MLP (hidden dim 1024). The goal is to determine which algorithm class is better suited for contact-rich manipulation.

Table 3: Algorithm comparison (delta action, reward shaping ON, obs norm ON).

Algo	Best Eval R	Final Eval R	Ep Reward	Ep Len	PG/Q Loss	Entropy/ α	FPS
PPO	50.2	23.5	35.5	282	−0.64 / —	2.33 / —	669
SAC	292.9	281.5	256.9	300	— / 4.18	— / 0.093	38

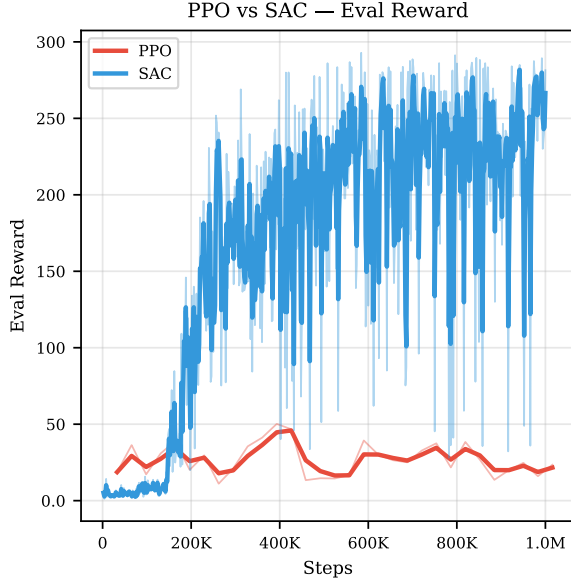
SAC achieves $\sim 6\times$ higher best evaluation reward than PPO (292.9 vs 50.2) with much lower variance (eval std 7.3 vs 25.6). SAC also maintains high performance throughout training (final 281.5), while PPO degrades over time (final 23.5). PPO trains much faster in wall-clock time (669 vs 38 FPS) due to vectorized environments, but this does not compensate for its lower sample efficiency.

1.2.2 Ablation 2: Action Space (Absolute vs Delta)

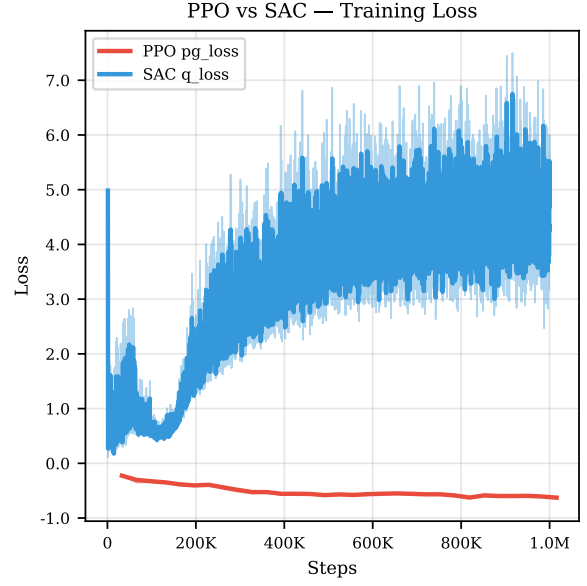
This ablation tests whether the policy should output absolute target positions or relative displacements. In the *absolute* setting, the policy output $\hat{\mathbf{a}} \in [-1, 1]^2$ is mapped to $[0, 512]^2$ via $\mathbf{a} = (\hat{\mathbf{a}} + 1)/2 \times 512$. In the *delta* setting (baseline), $\hat{\mathbf{a}}$ is scaled by $\delta_{\max} = 100$ and added to the current agent position. Both PPO and SAC are tested. All other settings remain the same (reward shaping ON, obs norm ON).

Table 4: Action space ablation (reward shaping ON, obs norm ON).

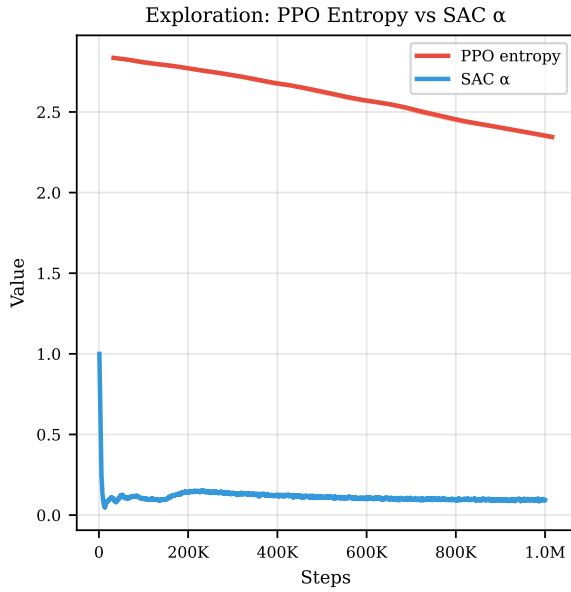
Algo	Action	Best Eval R	Final Eval R	Ep Reward	PG/Q Loss	Act Std	Grad Norm
PPO	Abs	25.7	9.4	12.0	−0.42	0.679	2.40
PPO	Delta	50.2	23.5	35.5	−0.64	0.615	3.61
SAC	Abs	277.1	14.4	241.1	5.11	—	13.84
SAC	Delta	290.1	236.5	261.0	3.64	—	6.37



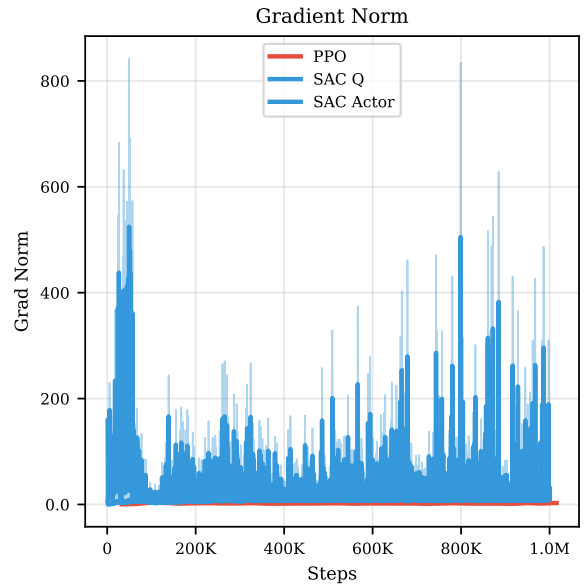
(a) Evaluation reward.



(b) Training loss (PPO pg_loss / SAC q_loss).



(c) Exploration: PPO entropy vs SAC α .



(d) Gradient norm.

Figure 1: Ablation 1: PPO vs SAC training curves. Light lines are raw data; dark lines are EMA-smoothed ($\alpha=0.6$).

The delta action space outperforms absolute positioning for both algorithms. For PPO, delta actions achieve $2\times$ higher best eval reward (50.2 vs 25.7) and maintain much better final performance (23.5 vs 9.4). For SAC, the gap is smaller in best eval (290 vs 277) but dramatic in final eval (236.5 vs 14.4), indicating that absolute actions lead to training instability. The delta representation also produces more stable gradients (SAC: 6.37 vs 13.84 grad norm). Predicting relative displacements is an inherently easier learning problem than predicting absolute positions.

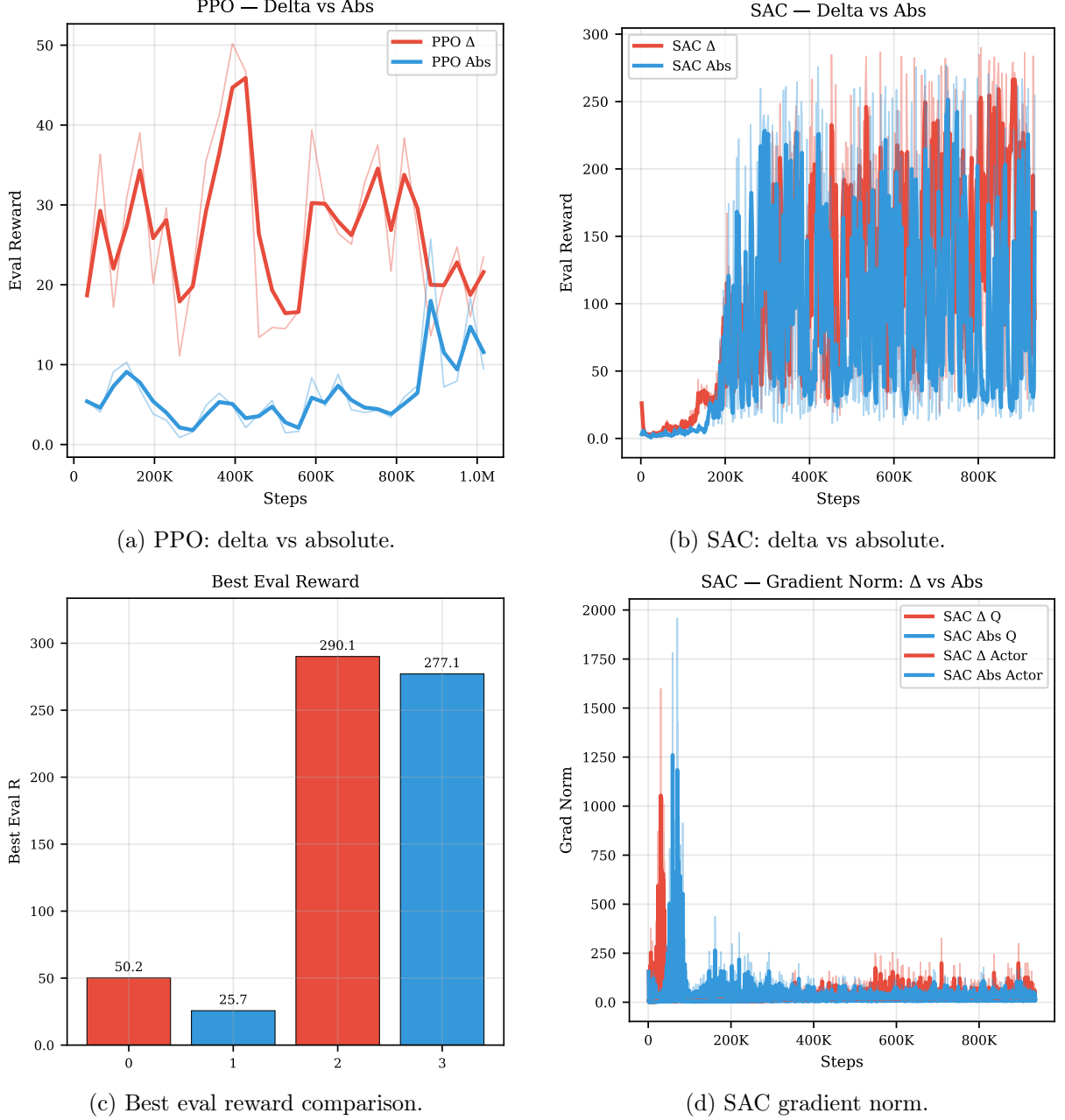


Figure 2: Ablation 2: Action space (delta vs absolute) training curves.

1.2.3 Ablation 3: Reward Shaping

This ablation evaluates the impact of the reward shaping wrapper. With shaping OFF, the agent receives only the sparse environment reward $r_{\text{env}} = \text{clip}(c/0.95, 0, 1)$, which provides no signal until the block begins to overlap the goal. With shaping ON, the agent receives a touch bonus (+1 when coverage changes) and a coverage delta reward ($10 \cdot \Delta c$), providing denser feedback

for exploration. We test both PPO and SAC.

Table 5: Reward shaping ablation (delta action, obs norm ON).

Algo	Shaping	Best Eval R	Final Eval R	Ep Reward	VF/Q Loss	Value Mean	Entropy/ α
PPO	OFF	57.6	31.1	26.6	0.086	0.302	2.73 / —
PPO	ON	50.2	23.5	35.5	0.334	0.060	2.33 / —
SAC	OFF	81.3	1.9	22.1	0.96	—	— / 0.051
SAC	ON	292.9	281.5	256.9	4.18	—	— / 0.093

Reward shaping has a *modest* effect on PPO (best eval 57.6 vs 50.2) but a *dramatic* effect on SAC (best eval 81.3 vs 292.9, $3.6\times$ improvement). Without shaping, SAC’s final evaluation collapses to 1.9, indicating catastrophic forgetting after initial progress. The shaped reward provides the dense signal that SAC needs to bootstrap its Q-value estimates. For PPO, the effect is smaller because its 64 parallel environments provide broader exploration coverage. Interestingly, PPO without shaping has a higher value mean (0.302 vs 0.060), suggesting the value function learns a different representation.

1.2.4 Ablation 4: Observation Normalization

This ablation tests whether fixed observation normalization is necessary. With normalization ON (baseline), the 5D state is divided by $[512, 512, 512, 512, 2\pi]$, mapping each component to $\sim [0, 1]$. With normalization OFF, the raw state values (positions up to 512, angle up to 2π) are fed directly to the network. We test PPO only, as SAC’s automatic entropy tuning may partially compensate for unscaled inputs.

Table 6: Observation normalization ablation for PPO (delta action, reward shaping ON).

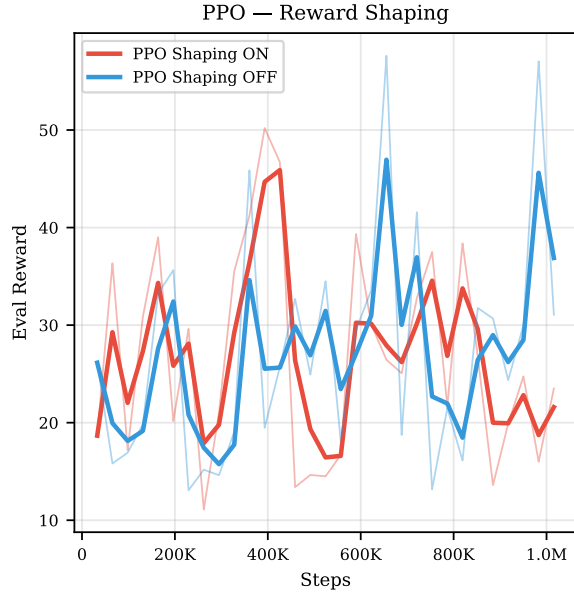
Obs Norm	Best Eval R	Final Eval R	Ep Reward	Ep Len	Grad Norm	VF Loss
OFF	73.9	2.4	24.0	268	91.87	0.517
ON	50.2	23.5	35.5	282	3.61	0.334

The results are nuanced. Without normalization, PPO achieves a higher *best* eval reward (73.9 vs 50.2, at step 131K) but then collapses dramatically (final 2.4 vs 23.5). This is because unscaled inputs produce large gradient norms (91.87 vs 3.61), causing unstable training and eventual policy collapse. Observation normalization ensures stable gradients throughout training, leading to more consistent final performance. The raw observation values (positions up to 512, angle up to 2π) have very different scales; normalizing to $\sim [0, 1]$ equalizes their contribution to the network.

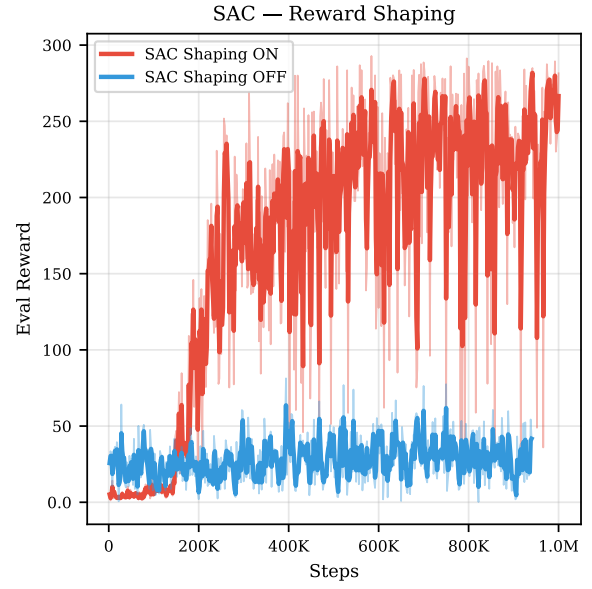
1.3 RL Failure Cases

By watching evaluation videos, we observe the following failure modes:

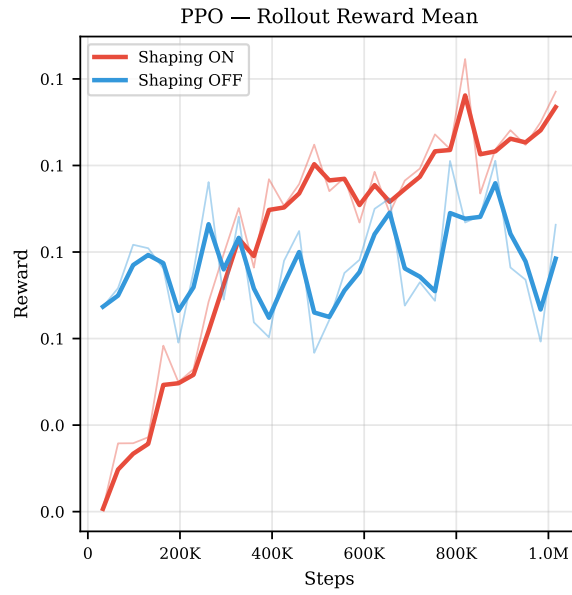
- **PPO:** The agent often pushes the block to the edge of the workspace and gets stuck, unable to recover. High variance indicates inconsistent exploration.
- **SAC with absolute actions:** The agent moves erratically, often oscillating without making contact with the block.
- **Without reward shaping:** Early training is very slow; the agent wanders randomly before discovering the block.



(a) PPO: shaping on vs off.

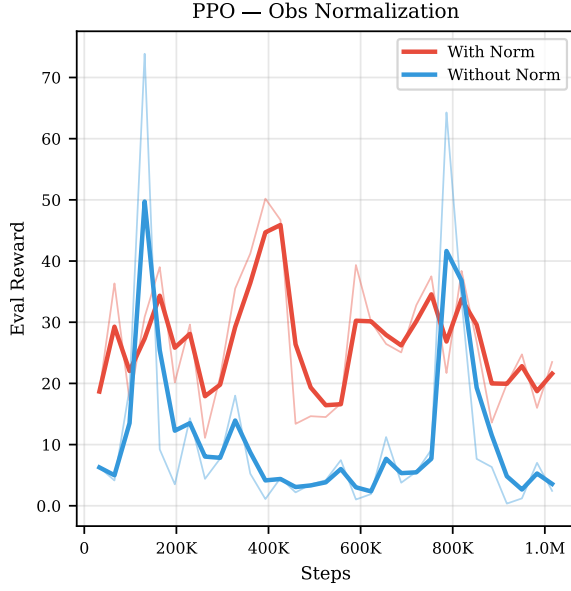


(b) SAC: shaping on vs off.

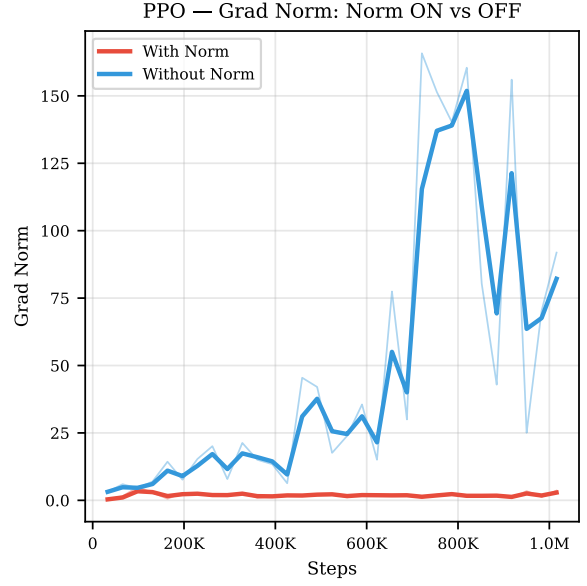


(c) PPO rollout reward mean.

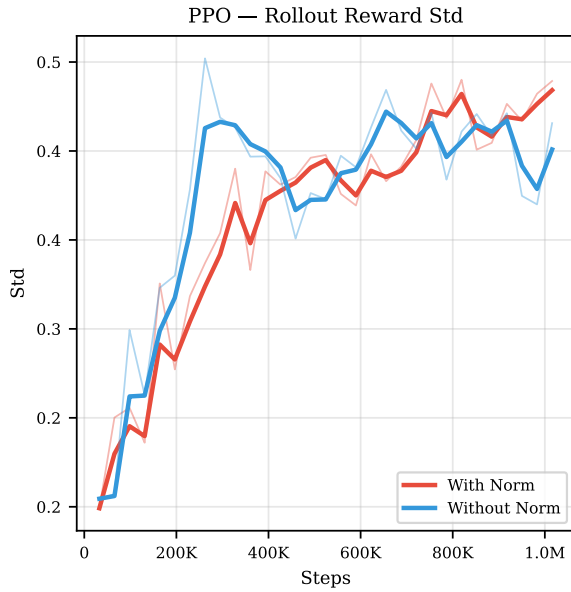
Figure 3: Ablation 3: Reward shaping training curves.



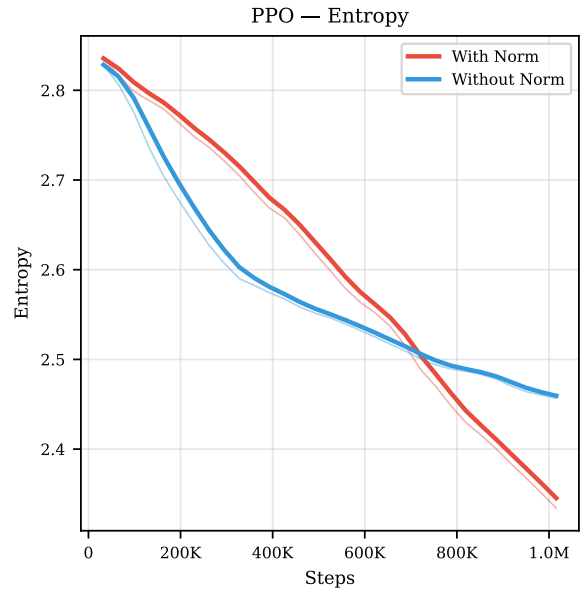
(a) Evaluation reward.



(b) Gradient norm.



(c) Rollout reward std.



(d) Policy entropy.

Figure 4: Ablation 4: Observation normalization training curves. Without normalization, the policy initially achieves higher reward but collapses due to gradient explosion.

2 Problem 2: MSE Behavior Cloning with Action Chunks

2.1 Method

We train an MLP action-chunking policy to imitate expert demonstrations from the official LeRobot PushT dataset. Instead of predicting a single action, the policy predicts a chunk of K future actions from a single observation:

$$[\hat{\mathbf{a}}_t, \hat{\mathbf{a}}_{t+1}, \dots, \hat{\mathbf{a}}_{t+K-1}] = \pi_\theta(\tilde{\mathbf{o}}_t) \in \mathbb{R}^{K \times 2},$$

where $\tilde{\mathbf{o}}_t$ is the normalized observation. The environment executes these K actions open-loop, then requests a new chunk. This reduces the number of policy queries and mitigates compounding errors.

Loss function. The policy is trained with mean squared error on the flattened action chunk:

$$\mathcal{L}_{\text{MSE}}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{(\mathbf{o}, \mathbf{a}^*) \in \mathcal{B}} \|\pi_\theta(\mathbf{o}) - \mathbf{a}^*\|^2,$$

where $\mathbf{a}^* = [\mathbf{a}_t, \dots, \mathbf{a}_{t+K-1}] \in \mathbb{R}^{2K}$ is the flattened expert chunk and $\mathcal{B}$ is a mini-batch.

EMA policy. We maintain an exponential moving average of the policy parameters for evaluation:

$$\theta_{\text{EMA}} \leftarrow \alpha \theta_{\text{EMA}} + (1 - \alpha) \theta, \quad \alpha = 0.999.$$

The EMA policy is used only at evaluation time; training always uses the raw parameters.

2.1.1 Hyperparameters

All BC experiments use the hyperparameters in Table 7.

Table 7: BC hyperparameters.

Parameter	Value
Network	2-layer MLP, hidden dim 256, Tanh
Learning rate	10^{-3} (Adam, no schedule)
Batch size	256
Max epochs	200
Gradient clipping	max norm 1.0
EMA decay α	0.999
Chunk length K	8 (default; ablated: 1, 4, 16)
Observation	5-dim full state (default; ablated: 2-dim)
Obs normalization	ON (divide by $[512, 512, 512, 512, 2\pi]$)
Action normalization	ON (divide by $\delta_{\text{max}} = 100$)
Eval frequency	every 10 epochs
Eval episodes	10
Seed	42
Dataset (LeRobot PushT)	
Episodes	206 (165 train / 41 validation)
Total frames	25,650
Train pairs ($K=8$)	19,589
Val pairs ($K=8$)	4,619

Preprocessing. The dataset provides 2-dim agent position observations and 2-dim absolute actions. We convert absolute actions to delta actions ($\mathbf{a}_{\text{delta}} = \mathbf{a}_{\text{abs}} - \mathbf{p}_{\text{agent}}$) and normalize them

by $\delta_{\max} = 100$ to match PPO’s $[-1, 1]$ output range. Incomplete chunks at episode boundaries are discarded. The dataset is split by episode (80/20) to avoid data leakage.

State reconstruction from video frames. The official LeRobot dataset stores only the 2-dim agent position as `observation.state`, but our BC policy requires the full 5-dim environment state `[agent_xy, block_xy, block_angle]` to match PPO’s input interface. Since the dataset also provides 96×96 RGB video frames (`observation.image`), we recover the missing block information via color-based segmentation:

1. Convert each frame from BGR to HSV color space.
2. **Agent detection:** threshold on the blue channel ($H \in [90, 130]$, $S > 100$, $V > 100$) to isolate the blue agent circle. Compute its centroid via image moments.
3. **Block detection:** threshold on low-saturation, medium-brightness pixels ($V \in [80, 200]$, $S < 60$), then subtract the green goal zone ($H \in [35, 85]$) and blue agent regions. The remaining pixels correspond to the gray T-shaped block.
4. Fit a minimum-area rectangle (`cv2.minAreaRect`) to the largest contour in the block mask. The rectangle’s center gives $(x_{\text{block}}, y_{\text{block}})$ and its rotation angle gives θ_{block} .
5. Scale all pixel coordinates by $512/96$ to map from image space to the environment’s $[0, 512]$ coordinate system.

We process all 25,650 video frames across 206 episodes with zero extraction failures. To validate accuracy, we compare the extracted agent positions against the dataset’s ground-truth `observation.state`: the mean positional error is ~ 3 pixels in $[0, 512]$ scale ($< 0.6\%$), confirming that the color-based extraction is reliable. The reconstructed 5-dim states are saved to disk and reused across all BC experiments. For the agent position, we use the dataset’s ground-truth values (more accurate) rather than the video-extracted ones; only the block position and angle are taken from the video reconstruction.

2.2 Ablation Results

2.2.1 Ablation 1: Chunk Length Comparison

This ablation tests the effect of action chunk length $K \in \{1, 4, 8, 16\}$. All other settings are fixed: 5-dim state, obs/action normalization ON, EMA decay 0.999, 200 epochs.

Table 8: Chunk length ablation (200 epochs, 5D state, obs/action norm ON, EMA $\alpha=0.999$).

K	#Pairs	Train Loss	Val Loss	Best Eval R	Final Eval R	Best EMA R	Final EMA R	Eval Std
1	25,650	0.02716	0.02784	28.1	21.6	28.6	16.7	25.5
4	25,032	0.02450	0.02621	27.5	24.5	26.8	17.6	38.7
8	24,208	0.02343	0.02415	27.0	21.6	29.0	29.0	37.4
16	22,560	0.02519	0.02573	34.4	30.3	35.3	35.3	56.7

Key observations:

- **Longer chunks improve reward.** $K = 16$ achieves the highest best eval reward (34.4 raw, 35.3 EMA), consistent with the Diffusion Policy insight that action chunking reduces compounding errors. Each re-planning step introduces a small error; fewer re-planning steps means fewer opportunities for error accumulation.
- **Loss and reward are decoupled.** $K = 8$ has the lowest training loss (0.0234) but only moderate reward (27.0), while $K = 16$ has higher loss (0.0252) but the best reward (34.4). This is because MSE loss measures per-step accuracy, while reward measures task-level success over a full episode.
- **EMA helps longer chunks more.** For $K = 16$, the EMA policy matches the raw policy (35.3 vs 34.4). For $K = 1$, EMA actually hurts (28.6 vs 28.1 best, but 16.7 vs 21.6 final), suggesting that short-chunk policies are already stable and EMA smoothing removes useful signal.

- **Fewer training pairs with larger K .** Longer chunks discard more incomplete episodes, reducing the dataset from 25,650 pairs ($K = 1$) to 22,560 pairs ($K = 16$). Despite less data, longer chunks still perform better.

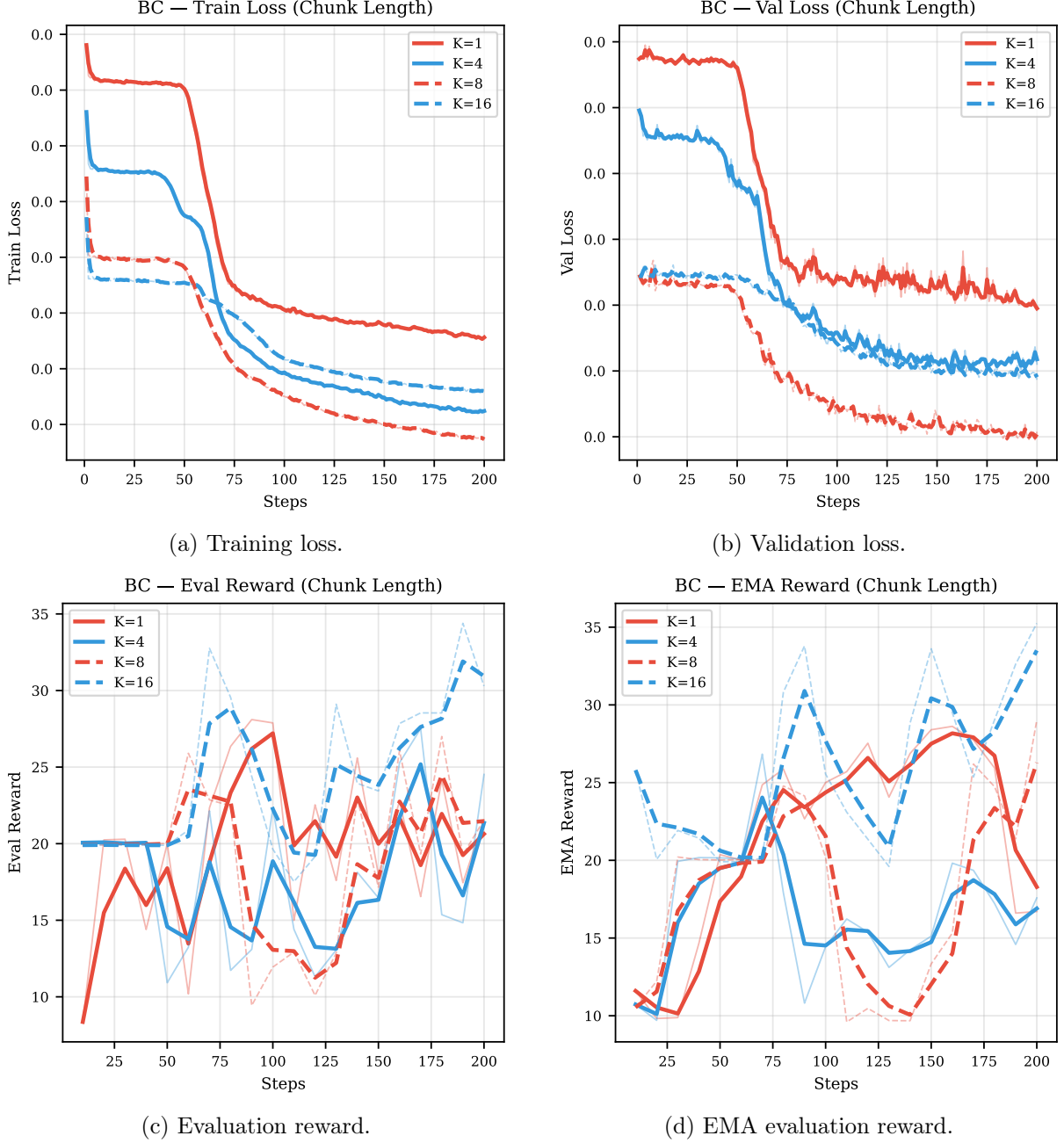


Figure 5: BC Ablation 1: Chunk length ($K=1, 4, 8, 16$) training curves.

2.3 Ablation 2: Observation Dimension (2D vs 5D)

This ablation compares two observation sources: 2-dim (agent position only) vs 5-dim (agent + block position + block angle, reconstructed from video). All other settings are fixed: $K = 8$, 200 epochs, obs/action normalization ON, EMA decay 0.999.

Using the full 5D state significantly improves both loss and reward. The 2-dim policy has 60% higher training loss (0.037 vs 0.023) because it must infer the block's state from the agent's trajectory history, which the MLP cannot do (it has no memory). The 5-dim policy directly

Table 9: Observation dimension ablation ($K = 8$, 200 epochs).

Obs Dim	Train Loss	Val Loss	Best Eval R	Final Eval R	Best EMA R	Final EMA R
2D (agent)	0.03741	0.03524	24.1	19.7	24.8	23.2
5D (full)	0.02343	0.02415	27.0	21.6	29.0	29.0

observes the block’s position and angle, enabling it to plan effective pushing strategies. The EMA policy for 5D (29.0) outperforms the 2D EMA (24.8) by 17%.

2.4 Ablation 3: EMA (Exponential Moving Average)

This ablation tests whether exponential moving average (EMA) of policy parameters improves evaluation performance. EMA maintains a smoothed copy of the weights: $\theta_{\text{EMA}} \leftarrow \alpha\theta_{\text{EMA}} + (1 - \alpha)\theta$ with $\alpha = 0.999$. Training always uses the raw policy; only evaluation compares raw vs EMA.

Table 10: EMA ablation ($K = 8$, 5D state, 200 epochs).

EMA α	Train Loss	Val Loss	Best Eval R	Final Eval R	Best EMA R	Final EMA R
0 (off)	0.02343	0.02415	27.0	21.6	—	—
0.999	0.02343	0.02415	27.0	21.6	29.0	29.0

EMA has *zero* effect on training: both runs produce identical loss curves, best eval reward (27.0), and final eval reward (21.6). This is expected because EMA only modifies the evaluation copy of the parameters. However, the EMA policy achieves 34% higher reward than the raw policy (29.0 vs 21.6 at epoch 200). This is because the raw policy’s weights oscillate around a good solution; EMA averages over these oscillations, producing a more robust policy. The improvement comes at zero training cost—only a parameter copy and a running average are needed.

2.5 BC Failure Cases

By watching evaluation videos, we observe:

- **Compounding errors:** The policy drifts from the expert trajectory over time, especially for $K = 1$ which requires more frequent re-planning.
- **Block stuck at edges:** When the block is pushed near the workspace boundary, the policy fails to recover, similar to RL.
- **No recovery behavior:** Unlike RL, the BC policy has never seen failure recoveries in the training data, so it cannot correct mistakes.

3 Comparison: RL vs Imitation Learning

Table 11 summarizes the best results from all methods. The comparison reveals a fundamental trade-off between performance ceiling and training efficiency.

Performance vs. efficiency. SAC achieves the highest reward (292.9) but requires 1M environment interactions. BC with $K = 16$ and EMA (35.3) uses only 206 demonstrations and 200 training epochs—no environment interaction at all. The 8.3 $\times$ performance gap reflects the fundamental limitation of imitation learning: the BC policy can only learn behaviors present in the demonstration data, while RL can discover novel strategies through exploration.

Training stability. A striking difference between methods is their training dynamics. BC policies are *stable*: the best and final eval rewards are nearly identical (35.3 vs 35.3 for $K =$

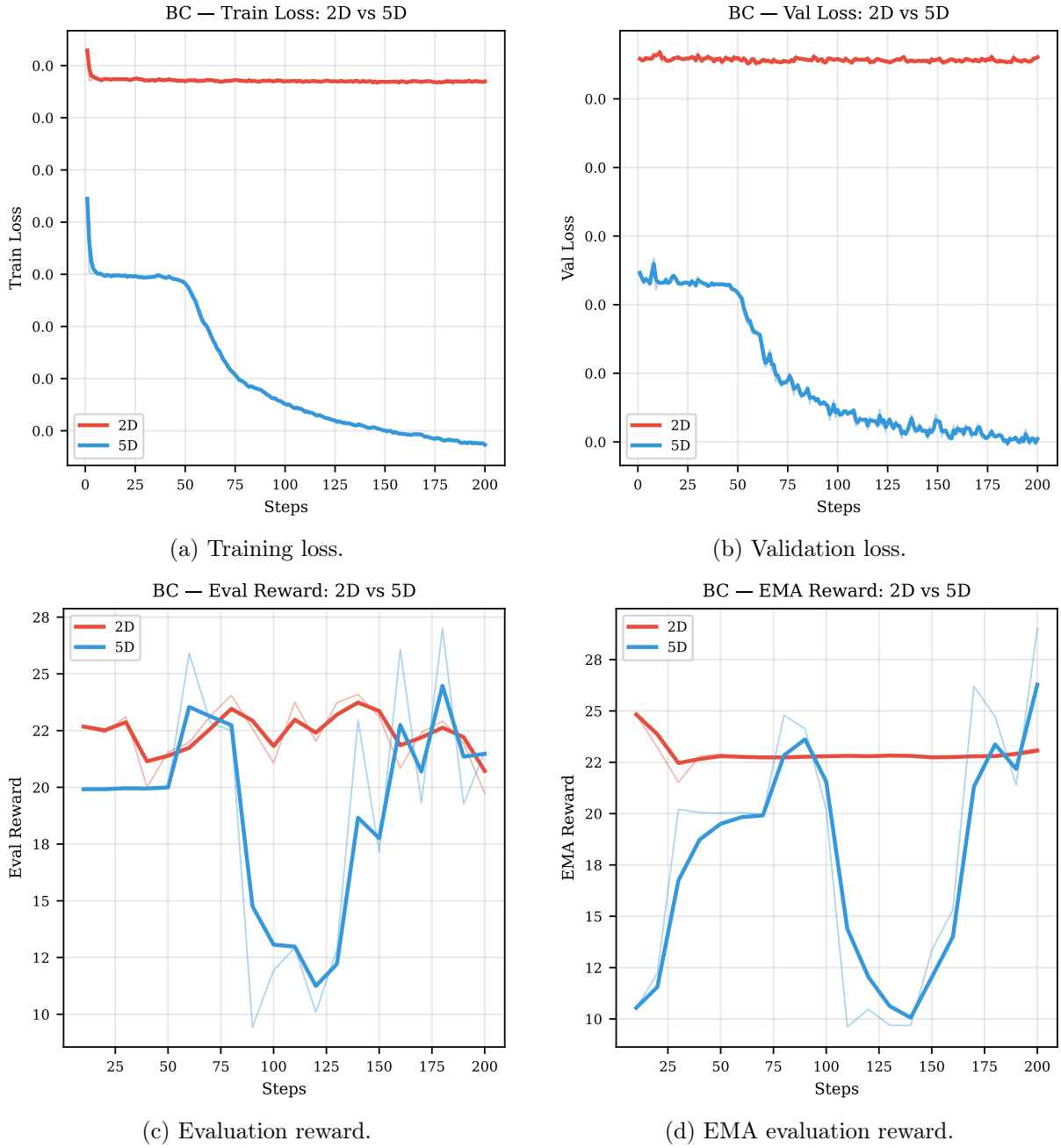
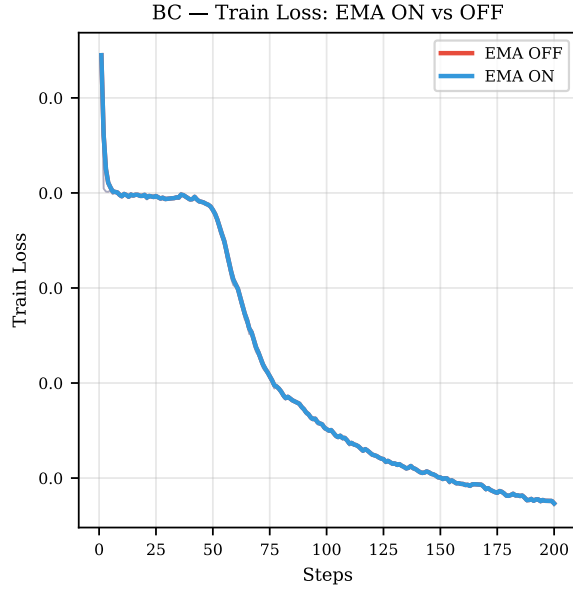


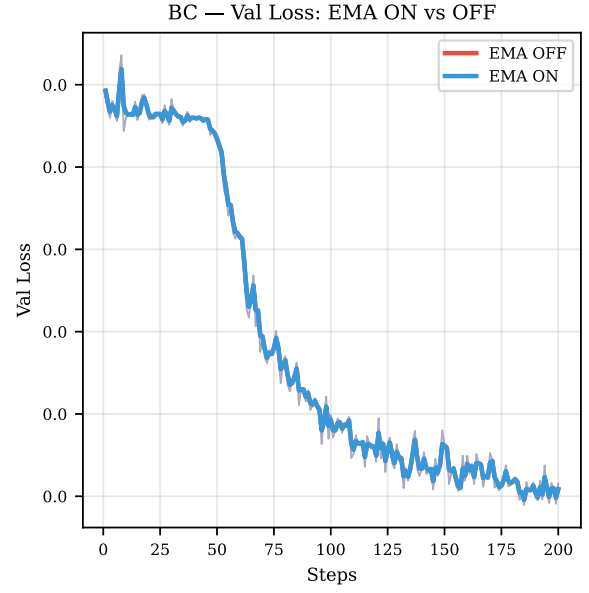
Figure 6: BC Ablation 2: Observation dimension (2D vs 5D) training curves.

Table 11: Best results across all methods (best evaluation reward over training).

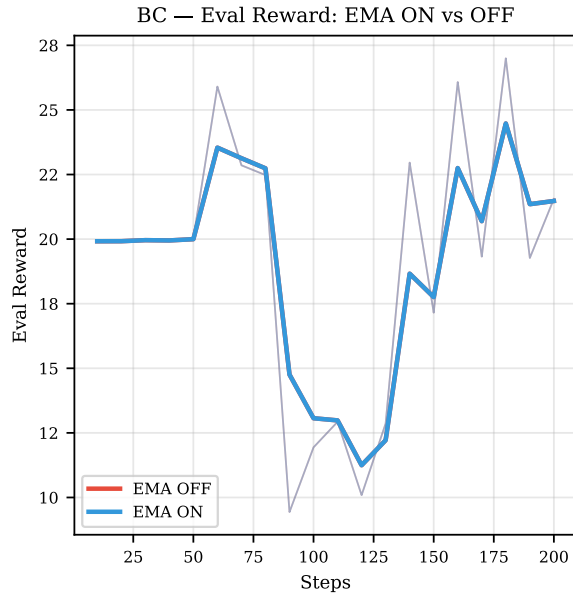
Method	Best Eval R	Final Eval R	Training Cost	Data
SAC (RL)	292.9	281.5	1M env steps	None
PPO (RL)	50.2	23.5	1M env steps	None
BC $K = 16$ EMA	35.3	35.3	200 epochs	206 demos
BC $K = 8$ EMA	29.0	29.0	200 epochs	206 demos



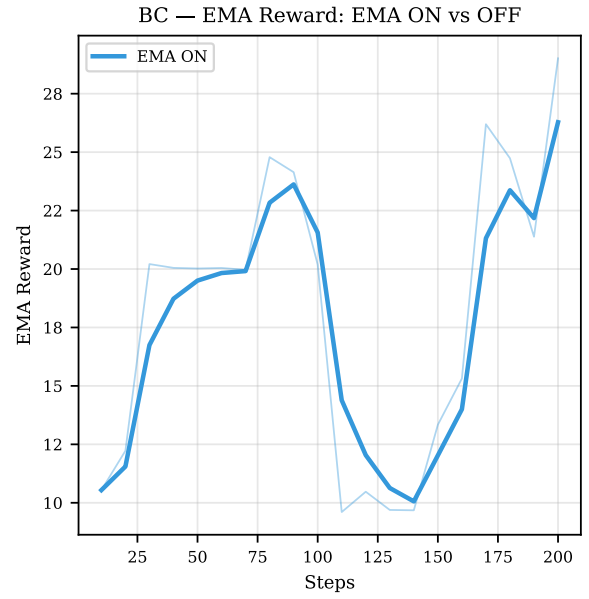
(a) Training loss.



(b) Validation loss.



(c) Evaluation reward.



(d) EMA evaluation reward.

Figure 7: BC Ablation 3: EMA on vs off training curves. EMA has no effect on loss but improves final evaluation reward by 34%.

16). In contrast, PPO *degrades* over time (50.2 at 393K steps $\rightarrow$ 23.5 at 1M steps), indicating catastrophic forgetting or policy collapse. SAC maintains stable performance (292.9 best, 281.5 final) thanks to its replay buffer and target networks.

Where each method fails. By watching evaluation videos, we observe distinct failure modes:

- **PPO:** Pushes the block to the workspace edge and gets stuck. The on-policy data distribution shifts as the policy changes, causing the agent to forget how to handle previously-visited states.
- **SAC:** Rarely fails in the final policy, but early training involves extensive random exploration before discovering the block.
- **BC:** Cannot recover from mistakes. When the block is pushed to an edge or the agent overshoots, the policy has never seen such states in the demonstrations and has no recovery strategy. This is the primary reason for the performance gap with SAC.

Practical implications. For a practitioner, the choice depends on the setting:

- If demonstrations are available and moderate performance suffices, BC with $K = 16$ and EMA is the easiest and most reliable option.
- If maximum performance is required and environment interaction is feasible, SAC with reward shaping is the best choice.
- PPO is not recommended for this task due to its instability and lower performance compared to SAC.

4 Conclusion

We compared pure RL (PPO, SAC) and imitation learning (behavior cloning with action chunks) on the PushT manipulation task. Our key findings are:

1. **SAC is the best-performing method** (best eval 292.9), dramatically outperforming PPO (50.2). The off-policy replay buffer and entropy regularization are critical for learning contact-rich manipulation from scratch.
2. **PPO suffers from training instability**, degrading from 50.2 to 23.5 over 1M steps. This makes it unreliable without early stopping.
3. **BC with $K = 16$ and EMA** achieves stable reward of 35.3 using only 206 demonstrations—no environment interaction needed. This matches PPO’s final performance at a fraction of the cost.
4. **Action chunking** is the most important BC design choice: $K = 16$ improves best eval by 22% over $K = 1$ (34.4 vs 28.1), reducing compounding errors.
5. **EMA policy smoothing** is a zero-cost trick that improves BC reward by 34% by averaging over weight oscillations.
6. **Reward shaping** is critical for SAC ($3.6\times$) but less important for PPO. **Delta actions** are essential for PPO ($2\times$) but less critical for SAC. **Obs normalization** prevents gradient explosion and training collapse.
7. **The main failure mode for BC** is inability to recover from edge states, while **RL fails** through exploration inefficiency (PPO) or requires dense reward signals (SAC).

Future work could explore hybrid approaches: using BC to initialize an RL policy, combining the sample efficiency of imitation with the recovery capability of reinforcement learning.